

Introducing ActiveInference.jl, a julia package for simulating active inference models and fitting t

Source: [YouTube](#) **Playlist:** 4th Applied Active Inference Symposium (2024)

Duration: 58:34 | **Uploaded:** Unknown **Transcript method:** auto_caption

hello we're back with Samuel ner Jonathan Larson and Peter Wade discussing active inference djl staying with the active inference plus Julia theme thank you all for coming on and sharing this work so looking forward to [Music] it hi everyone it is a pleasure to be here so like Daniel said we'll present um a new package for doing active influence simulations but also for applying it to data in Jia and we will leave time for questions at some moments during the presentation and then sometime in the end additionally um we're three people because we've been working on this together so we'll alternate a little bit with the the all right let's work the first uh it's nice just to mention who we are so we're all at o University I'm a PhD candidate and Sam and Jonathan are students who have been working on developing this package with me and we all come from computational cognitive science specifically it's called cognitive modeling you Model Behavior with competition models and of course within that specifically predict presing models H including active influence models and that's why we made this package um and we use this in the context of competition Psychiatry for clinical questions and also more broadly for just modeling behavior and this package is part of a somewhat larger um process of project of developing a software for these kinds of research questions in Julia and I'll start with just giving a brush up on what cognitive modeling is in general so some members of the audience might be familiar with this but others might not um just to set the stage for an introduction to active inference you all have seen this before so we'll go through it briefly but it's nice to have this finger tips then we'll get to the meat of the matter which is of course how to do this specifically in code with active inference JL and both simulating behavior and also fing into data and finally if there's time we have somec and some future work that we'll be happy to spend some time we might skip that as well all right cognitive modeling or computational cognitive modeling um comes out of psychology C to science where the main question in those fields is to find out what are the mental mechanisms that makes us do the things we do and so the classic perception action Loop we have to pick here people maybe be

familiar with that too it's going to be our AC book where you have some agent which could be a person but it could actually also be an animal or even an organoid or even a simulated thing system that receives inputs from its environment and takes actions that change its environment and we are here interested in essentially and I don't know if you can see my cursor but essentially the relation between those observations and the actions that are created so basically the part after the observation error ends and before the action error starts and traditionally in Psychology and cognitive science this has been done experimentally so you take people out of natural situations and controlled laboratory situations where you measure well you control both so you decide what they observe and you measure what they do and so the kind of classical example of this is have a computer screen you show people some image or something maybe you're asked them to classify them or predict what's going to happen and you measure what they do and then you have a measurement of inputs and measurements of outputs and you then create some theory that will make some prediction as to how they would behave and of course this is a big field you might have theories of personality types where you predict that actions will change faster depending on inputs there's theories of reinforcement learning Etc and so importantly here we're using already Natural Science approaches experimental approaches with quantitative analyses but still with purely conceptual theories so the theories are not formal as they are from let's say physics you have mathematical theory and the trick is often to go from some conceptual Theory to a Quant prediction that you can then test and specifically give me a moment here sorry about that and specifically as many people know there's been a what's called a replication crisis in Psychology and cognitive science where the whole issue is that the experiments don't replicate and that's partly due to statistical methods issues but also partly due to problems stemming from the fact that our theories are not clearly formulated so one sentence that's saying that there is social learning might mean many different things and so a Recon attempt to address them is to move into formal theories just like in physics so computational models of this again relation between observations and action and this is inspired by physics so in physics people are no strangers to having things they can observe in our case it's the behavior but in physics it can be let's say radiation from the Sun and then trying to make inference about something you can't observe in our case the mind and this case perhaps the molecules of the of the Sun and so the methodology is exactly the same create mathematical equations that will describe how CS evolve over time given observations in the environment and test them on how well they describe empirical collected actions observations and here I

just put on one of the very classic cognitive models which is the scholar wagner where you have some very very simple equations that describe how actions depend on the observations in the previous time step with some learning and a classic learning data and that's to say that the many the first C models were very simple in a few equations you would kind of describe these. Rel I think primarily in three fields is this approach used where you have some person or some system and you want to find formal models of its behavior and that's in computation Psychiatry where you want to use it to understand psychiatric conditions mathematical psychology which traditionally has been broader it's essentially the same questions but a different community and different types of models used but also broader just behavior in general not specifically Psychiatry and computational Neuroscience where you want to relate these um models to for example neurobiological processes that could implement and they also overlap like a overlap with artificial intelligence and Eng engineering and so on where all right so I think a primary use case especially in our field of active influence is theoretical simulations where you just take some phenomenon let's say learning or let's say learned helplessness you create a simulation that can do that uh create that kind of behavior and you suggest this as a proposed mechanism of how that behavior comes about you can use this for one agent doing one type of behavior or you can use it for many and then you're doing agent based models uh specifically cognitive agent based models where you rely on actual theories to but you can also apply it to behavioral data and here you're often trying to either find out what mechanisms provide actually observed data you might want to distinguish let's say clinical populations from each other and what's called computational fingerprinting you might also use results here to inform future theoretical simulations so you can have abms where every agent has parameter values you found an experiment and you can also use this to design future experiment where you use simulation to make sure that your experiment design will give you as good results as possible and finally you can also apply these approaches in machine learning and engineering and here of course you can make robots that use cognitive models that people might also use in the way that in general technology and knowledge of biological systems have always work with each other so in the same way you might be inspired how to build a plane by looking at a bird you might also be inspired on how to build an image recognizer by looking at human being our packages specifically Target the application to behavioral data so they can do theoretical simulations and also in principle be used for engineering purposes but we've optimized them for this situation where you have participants you put them in an experiment you get behavioral data and you use those models to

understand that data and that's um what distinguishes it from some of the other software tools that are present um so PP now has this capability also and but otherwise many of you and there is in the matlab SPM software also some ways to apply to data otherwise the focus has often been on theoretical simulations or on machine learning and if you look at packages like R and fur which is incredibly powerful and well made um that also at least to my understanding focuses more on the machine learning engineering part where the ultimate goal is to use these models to build tools that can do things um or maybe for simulations but not to specifically apply to behavior all right um and I'll just give a sense of what the workflow here looks like because if you come from theoretical simulations or engineering this might seem really well to start off designing hypothesized models for how people work you then check these models before applying them to anything but whether they are worth applying so you do what's called parameter recovery and model recovery where you first simulate behavior with some parameters you then estimate those parameters and you see if you can actually estimate the real parameters if you can't recover parameters that you know are true you can't trust this in when apply to real data and also prior and posterior predictive checks where you just check that your model can actually generate the kind of data that you might see in an experiment and if you can't it's also not where the clients will be able then you fit it to data which means uh finding out what parameters of my model what they must have been to generate the data that I did so In classical models it would be learning rates we ask oh this person adapted this fast so his learning rate must be this and this in the context of active influence it's usually um parameters of the generative model the A matrix and B matrices for example that we want estimate and finally we can also estimate how well the model fits our given data because some models might describe some data better than others we might want to use this to select we use this to relate parameter estimates to some measure that's a cross participant often this is clinical populations we might say our learning rates or are A matrix parameters different between clinical populations but it might also be relations to let's say age or psycho pharmacological interventions you can also relate it to within participant things let's say physiological correlates and could be pupation Etc but the primary use case of course is neuroimaging where you say oh according to my model he now this participant now has a high prediction error do I see a difference in brain activity at this moment compared to when he has low prediction error this often to find out what the neurobiological implementation of the computations of think people do and finally um you might compare models and ask questions like which of these hypothesized models are the ones that best describe the data and

therefore perhaps the actual mechanism within people and so you could compare an active inference and reinforcement learning model or let's say two active inference models with different generative models inside finally it's worth mentioning that fitting this to data there are many different approaches um broadly speaking they are basion and frequency to methods they have different advantages and disadvantages we usually use basian approaches and that's a long discussion of why that's nice they propagate uncertainty they're good you don't have so much data but I W go into that just to say that there are also other ways to fit models to data and within basion methods because you can't do exact base to fit them there are also approximations one classical approximation is very methods where you choose some arbitrary distribution we call it q and you make it be as similar as possible to the true posterior belief about the so the optimal parameter estimate you do this by minimizing a variational re energy and as it happens that's also what active INF agents use another in cognitive modeling widely used method is sampling approaches like Mark of chain C there are also others where essentially you try out many different parameter values and for each of them you calculate the likelihood of that value you do this in a smart way but it still takes a long time and you get a full estimate of the probabilities of different par values they have different claims to being useful for different um use cases so assembling is good um for parameter estimates but they take a long time they might Nota work variation base has some claims to being a better estimate of quality of fit or model evidence there's a whole few where people are still writing and if anyone's interested we can take questions on that but importantly the tools we make here can do either these two finally it's just worth mentioning that we as researchers use ban methods to understand people's behavior but within active influence we also theorize that people use basian methods to understand the world it's nice to keep those separate so we might use a sampling approximation to Patient influence to fit our models to data but still believe that people use variational Bas to understand their environment and so in the following when we talk of generative models we're not talking about our model of people we're talking about people's models of the world and again do ask questions that that's one of the tricky points sometimes when we're doing this what's called metapas good there are many paradigms within cognitive modeling mathematical psychology have many many ways of modeling often specific behavioral tasks there are these old cognitive architecture models actr and sore are biggest ones where people also trying to do unified models of behavior in general which rely on very different assumptions than active inflence and often we're not aware of them perhaps we should be so I'll just bring up here and they are kind of raw old school computationalist

models different modules interacting there of course reinforcement learning models where you assume that people do actions get rewards from their environment and try to find actions that maximize those rewards there many others like deep learning models Etc and but of course today most relevantly there are Bayesian mind models you think that people's minds use Bayesian influence of some sort to understand the there are many schools of Bayesian mind models might look at just ten Bayes for a different School than ours but active inference specifically belongs to a variational Bayesian mind model we use variational approximate Bayesian inference as a model of how people work and this has close ties to predictive coding and predictive processing so broadly speaking we might call this rational Bayesian predictive processing models they can be just a perception like Hoyer filters or plenty of models out there or they can include action as well and when they include action it's called active INF so variational Vision predictive processing models of perception and action is active and and now we'll give an introduction to that but we'll leave time for one question before we move on are there any questions here I'll ask one maybe you'll revisit it later to answer asked what's the biggest challenge you faced building this package that's a long and Technical discussion I think the main thing is that in Julia there are many ways of doing what's called the automatic differentiation which is a technical thing that you need to do the parameter estimation and there's just a whole ecosystem of packages doing that and we've had to become very familiar with those to find the ones that work the best and and that's not an easy task the part of the package is to hide that from the user so they don't have to engage with it I don't know if you agree with that probably yeah we can revisit it okay all right one more question just while we're while we're pause Frasier asked is there anything in active inference JL that is particularly suited to compositional agents or collectives kind of leading you into your paper I know so I'll return to that in the end if we have time but um basically you can use the same method that we use to make inferences about a given human participant in an experiment on instead of a human participant you can use it on a simulated Collective and treat that Collective as if it that is also doing some kind of cognition or active inference at least which is of course fundamental to active inference in general we expect a cell to do active inference but also person and also Collective and so if we treat a simulated Collective as let's say a foreign system that's doing something we don't know what is we can fit a model to that in exactly the same way we do to a person and make inferences about let's say the parameters of that simulated collectives D model of the mind and in the end time I'll just show like results from that paperwork we do exactly this just demonstrating it example that's an exciting question

obviously all right all right I will just briefly uh touch on the software ecosystem um that we build the package in um so firstly why jya uh it's fast and it's very intuitive meaning that if you come from python or matlab you can carry a lot of the same inations um over into Julia and then it has a community that is growing quite fast of people doing data science machine learning and Caron modelings we're doing then Julia is natively auto differentiable which means that you don't need to have a back end in another language to do auto differentiation uh which makes it very fast in Julia for this there's a very uh nice package package called touring. JL uh which is just a package for probabalistic programming in general and it can do both uh sampling methods such as mcmc or no uturn sampler um and and variational inference uh as well then in Julia there's also a sister package to active inference which takes different kinds of cognitive models and make it very easy to fit data and to those kind of models uh by kind of like wrapping touring uh and that's where kind of like our active inference JL package comes into um play as this is a cognitive model that through action models can be used uh to touring to fit uh data quite easily and we'll show later in the presentation as well then we'll just briefly touch on some of the core elements of active inference uh with regards to perception action and learning so for the perception part this is uh the minimization of the quantity variational free energy where we through this uh minimization we can infer States optimally based on observations and there's different kinds of variational inference or sorry different kinds of algorithms um that we can use to do this uh I'll later go into the specific algorithm that we're using uh in this package then we have action and active inference which is Guided by the minimization of a quantity called expected free energy and this can have a have a lot of different kind of Expressions if you look at the ones that we show here in the slide we have one that's Information Gain versus pragmatic value which is basically where you get the Explorer exploit behavior for free in active inference which is really nice um and then we also use uh the expected free energy to calculate a posterior probability over policies uh where policies are actions so this is kind of like the quanti that we use to kind of like guide the action selection uh we're also going to go into that a bit later then we have learning in active inference this is uh specifically for categorical distribution beliefs so there's other ways of learning for continuous um cases however here we take it for categorical distributions uh for this we use derish lay Prius over the generative model parameters this is basically derish lays they have concentration parameters that we can then add counts to and this is the update equation that we show here where we have an Omega which is the forgetting value times current concentration parameter and then you add the new here is the learning rate and then

you then add the k_i which is the data observed and in this sense you just add counts to the concentration parameters and then through normalizing it uh it becomes your new updated uh generative model parameter um we will come back to how that's implemented in the package as well and then uh a quick refresher on partially observable Markov decision processes so we just want to make sure to say that this is one type of generative model uh in active inference there's other types of generative models as well and uh pmpd they are not active inference specific um in the sense you can also use it in other Frameworks um and then this uh what we have done in this package is a discrete State space uh implementation of pmpd you can have continuous State space implementation of pmpd uh pmpd is the moving element of it though is that the time steps they are discrete and not continuous um so we'll just briefly go over conceptually uh the different parameters in the pmpd and for this uh we have created sort of like this little um toy environment uh to explain this and if you can see this thing here where you kind of like if you imagine that it's a dial uh where we can spin it in different ways uh and then the goal is to get state three on top in the little rectangle um so then you have three actions you can spin counterclockwise clockwise or you can just make it stay in place which you would do if you have uh state three on top for example um but we'll start with the observation State likelihood model basically what this encodes is the agent beliefs on how States generate observations so it's a categorical distribution where each column uh has to sum up to one so in this state uh in this case as we have a identity Matrix uh this indicates that the agent can uh determine stochastically infer uh the state based on the observation so if the agent observes observation one then it will infer that it is in state one and this can be more or less entropic in the sense we could also have a completely uniform uh column where we just have $1/3$ for all of these and then the agent wouldn't be able to infer the states based on the based on the observation all right so this was the observation State likelihood model then we're going to go to the state transition likelihood model and this is basically how the agents believe beliefs that the states change over time um so in this case I've taken the example of doing it for the action of going counterclockwise because this is usually dependent um on actions so here we see that for example if the agent is in uh State one and it does the action spin counterclockwise then it believes that the next state it's going to be in uh is going to be state two and the same if it's in state two then it believes that it's going to be in state three um so that's the state likelihood uh transition model then we have the prior preference and so this is basically where we kind of like get to encode uh kind of like the preferences or the goals of the agent Um this can be done both Over States and observations but in our case we have done it for observations and this is the case in our package as well so

here for example for observation one we have encoded a dislike uh for this observation which is coupled to the state one and then for observation two we have neither dislike nor preference and then for observation three which is uh coupled um to state three which is kind of like the goal that we want we have set a preference and this is important uh as this kind of encodes the pragmatic value um which is then part of the calculation of the expected free energy as you can see here below um yes and that's the C parameters then we'll continue to the D parameters which are basically just the agents initial belief at time one uh which state is it going to be in and here we just encoded to be certain that it's going to be in state one then we have the prior over policies you'll just uh quickly introduce the notion of policies where policies are collections of actions that can have a specific L length determined by the policy length which is basically how long in the future how many time steps in the future should the agent uh plan so for this case we only have one time setep in the future which is why we only have the three actions here and this is also sometimes described as the agent's kind of habits meaning that it has a preference for certain actions in this case we have given it a uniform distribution meaning that the agent doesn't really have any preference over uh any policies all right so now that we have the generative model down uh I'm going to show you how to implement this in our package um and we're going to do this um by providing a worked example of a teammates environment where the agent is going to learn the reward probability in the arms and for this I'll just quickly have to introduce the notion of factors and modalities so factors uh are basically different kinds of States so in this case it would be we have location States and we have reward condition states where reward condition states are whether the left arm or the right arm is going to provide the reward then we have modalities um modalities are basically different kinds of observations um so here we have a location observation we have a reward observation and we have a queue observation and I'll go into these uh when we get to Define them uh in the generative model to start using our package you can just add it from the Julia registry using the Julia package man manager uh as shown here um basically for this code you can run through it uh by yourself and it should be able to run um then we initialize the package and we also import the environments module which is important uh for the teammates environment that we're using as an example here and then we initialize the tmas environment and here we just set a 95% probability of providing reward in the reward condition arm so now we can create uh the actual generative model uh the pmdp generative model and you can do this manually by yourself um but then you have to make sure that the generative model parameters are typed in the correct manner uh which is shown here where it have to be vectors

of rays for ab and vectors of vectors for c and d and Vector of floats for E uh it's very similar to object arrays in numpy however we have created a helper function that makes this uh a lot easier and so I'll just quickly go through the arguments that the helper function uh takes so the first argument um is um the number of states in each factor um so remember we had two factors we had the location State factor and the reward condition Factor where we had in the location factor we had four states we had the sense the Q location and the two right arms so that's four and then we have two reward condition States reward condition right and reward condition left um so we take this as a vector and give it uh to this function as the first argument um then the second argument is the number of observations in each modality so we have uh four we have three different modalities in this case where we have four observations in the first modality the location modality we have the location sensor Q and The Arms then we have uh the second modality which is the reward modality it has three observations it has the observation of no reward reward and loss uh um then we have the last observation modality which is the cure observation modality this we also take as a vector and give to the function as the second argument the third argument is a number of controls for each of these factors so we have two factors the location factor and the reward condition Factor um we only want the agent to be able to control the location factor so it can move freely uh between the locations but in the reward condition State Factor it shouldn't be able to control what arm should be the reward arm and so therefore we just give it one in this case then we have the policy link which I have described before it's how many steps in the future does the agent plan and then we have uh the fifth and last argument which is the initial fill which basically says like when we have cons when we have made the structure of these turn to model parameters uh what should be the initial filling of these and here it takes both uh it takes zeros which we have chosen in this instance and then also takes the argument random which is nice for testing and uniform uh which is the default uh in this case good so now we just uh having created uh this helper or these generative model parameters we just need to fill them with the appropriate values so we'll start out uh with the observation State likelihood model and remember this is the one that encodes the agent beliefs of how States generates observations and then infers States based on the observations it receives so in this case we have the location OBS observation which is not as interesting to us um basically it's an identity Matrix meaning that the agent just infers the correct State because we have encoded to do so um and then we have two matrices here because we have for each of the location states which is one Matrix and then we have two uh matrices one for each reward condition State Factor um if you go to the

second modality which is the reward uh modality this is the one that is interesting to us because if you look at the First Column and the last column this is the Q and Cent location we see that it expects that these are going to provide the observation of no reward however for the second and third column which are the right and left arms uh we see that in the second uh row that it has a 50% chance uh that it believes that this state is going to produce a reward observation and then if we go to the third row we see that it has a 50% chance of generating this or the agent believe that there's a 50% chance of generating this observation so in this sense the agent doesn't know which of the arms is going to provide the reward observation and this is the the modality that we want the agent to learn we wanted to approximate the actual reward probabilities in the tmas environment then we have um the Q observation modality which basically just tells the agent that only when it's in the Q location does it uh correctly get an observation that informs it on the reward condition state of the environment um and then we're going to add uh a prior over α and this is where the learning part comes in um the prior over α um is the dlay distribution and the way that we create this is basically just that we copy uh the structure of α and then we add a scaling uh concentration parameter which basically says that these counts in the DeRay distribution if they are very high like if we give it high scaling concentration parameter each uh new data observation is not going to matter as much if we set the scaling uh concentration parameter low then each data observation we have is going to going to matter a bit more and then it's important to say that and this is bit clunky but uh this is now going to define the α matrix so the uh the prior over α is going to define the α matrix meaning that the α matric that we just made is not going to be exactly used however we used it as a template to create the prior over the α um great so now we have the observation State likelihood model now we need to encode the state transition likelihood model and this is uh the matrices for the first uh Factor the location factor and basically we just note here that we have rows that are filled with ones meaning that from wherever the agent is for that action it goes to that place that it wants to go to um and then we have the second uh factor which is the uncontrollable one um which is the reward condition factor and basically this identity Matrix just tells the agent or the agent believes that the reward condition State uh doesn't change uh over time that it stays the same good so now uh we come to an interesting parameter which is the prior preference over observations so we're we have a vector here for each of the modalities however the one that we are interested in is the reward observation so in this case the first index is the index of the observation no reward and we said no preference or dislike for this the second index is the uh observation of reward which we said to have a

preference for that it wants the reward and then we have the third index which is the uh loss observation which we set a dislike for yes and then we continue to the D uh parameters uh where we have a vector for each of the factors we have the location factor μ where we basically tell it that it's going to start the first index is the center location then we're just telling it that it's going to start in the center location and then for the second factor which is the reward condition Factor we just have a uniform distribution meaning that the agent does not know what the reward condition uh is going to be good then we have the last of these parameters uh which which is the prior over policies and we're just going to hear provide it with a uniform U prior so this means that as the E parameters are the habits uh it's not going to have uh any habits in this case we see here that we have 16 different policies because we have a policy length of two and we have four actions so the total number of combinations of these two actions is going to be 16 μ and then as said we just give it a uniform distribution over these μ policies good so now we're almost ready uh to initialize the acted inference agent before we do that we have to specify some settings μ and I'll just briefly go into these what they do uh the policy length we have talked about the use utility uh basically means should we include the C parameter preferences that we just said and we've said this to true then we have the use States info gain which basically says that in the calculation of the expected free energies which is going to guide action μ should we include the expected Information Gain and then we have the used parameter info gain uh setting which basically says that when we calculate the expected free energy should we include the expected learning or updating of the generative model then we have action selection which which can take two values either stochastic or deterministic μ I'll go into what they do when we come to the function that actually uses the setting then we have modalities to learn μ which basically specifies what modality do we want the agent to learn μ in our case we want wanted to learn the reward observation modality so we specify this by giving it two we also have factors to learn uh this is only relevant when we're doing B and D Matrix learning so it's not going to be relevant in our case then we have the fix Point iteration number of iterations and the fix Point iteration uh tolerance which is basically saying that μ when should the inference algorithm stop so you can set it to either as we do here after 10 iterations or when the difference between iterations are equal to a lower between this threshold yes the last thing that we need to Define to initialize the agent is the parameters here I'll just briefly go over what they do the gamma is an inverse temperature function on the expected free energies uh then we have the alpha which is inverse temperature on the action uh selection and then we have the learning and forgetting

rates uh for the priors over a b and d uh parameters which are the ones that we currently have learning for the ones that we are interested in at the moment though is the prior over a uh which is the learning rate and forgetting rate for this which we're going to set to its default value of one and the the values here are all the default uh values good so now we can initialize our active inference uh agent by providing it with the generative model and the settings and these parameters and we're almost ready for the perception and action Loop in itself and before we do that uh we just have to specify how many iterations does this Loop need to contain and then we're at the teamas which just gives the agent like an initial observation and this is the perception action learning uh loop and it has five functions and I'm going to go through them and just briefly explain how they relate to active inference um so the infast states function is the one that actually does the minimization of the variational free action uh the variational free energy um and this is the perception part of active inference um here in our package we use an algorithm called fixo iteration also known as coordinate Ascent variational inference um and here we also just uh we provided a picture of an observation where you can see that we have these three observations the location observation reward observation um and the Q observation then we have the infer policies uh this is the one that actually calculates the expected free energies and it's affected by uh the settings that we talked about before uh and the gamma and the E parameter it Returns the expected free energies and it also returns uh the posterior uh over policies which is and it's this posterior probability that we are going to be sampling from when we are sampling actions um so basically this function sample action uh just samples from this posterior probability over policies um and it is affected by the action selection where it could be either stochastic or deterministic if it's stochastic it just samples from the probability distribution but if it's deterministic it all always takes the policy with the highest probability then we have the update a uh which is where the learning occurs and this is where we basically add the D lay counts uh to the prior over a and then it updates the a by normalizing the concentration parameters then we have the step teamas which is basically just the agent interacting with the environment and then it provides uh the observation uh for the next iteration uh in the perception action learning Loop um so this is the the loop um and this is the one that's is also going to be used uh in the model fitting part um and we are I think we're taking some questions now one question one question one question yes okay I'll ask one question from Tintin in the chat they wrote hello thanks all for this great work have you implemented basian model reduction or incremental model construction structure learning with active inference JL rather than manual construction no that's the short so U the package is

just now ready for deployment and there's many ways to extend it which we work on but also I mean if people are interested in collaboration to develop the package it will also be one thing so this is exciting for two reasons like in two ways by the way um basically model structure learning it can either be model structure learning from the perspective of the agent learning the structure of the environment but it can also be model structure learning from the perspective of us researchers wanting to find out what kind of structure a given agent has and both of these purposes something we would like to do I think a useful um kind of connective tissue question where would you put active inference JL in relationship to pmdp does in Python and what RX infer does in Julia so pmdp was originally um a port of the SPM functions into python brilliant work by con and colleagues where they make it modular and easy to use but made for theoretical simulations only and I think this summer they just finished Jack's reimplementation of pp which means now you can also use it to fit to behavior but it's not designed wasn't primarily designed for that and um it was also not made for engineering purposes python is too slow although I'm sure could in principle and since then has come the C++ package right um which is much faster and I think but hard to use because it's C although there are wrappers in different languages that would be useful for machine learning RX and Fir implements active inference and many other things on a style cognate which is a very efficient and Powerful way of thinking about inference and variation about inference and that is optimized much more for engineering if you want to build something that does something using active inference go C++ or um or RX info if you want to apply to behavioral data then go PDP or active inf. JL um and there are some tools for this also in metl but I think our package and mdp are made for to make this easier and as you might notice we use much of the syntax from PP so we're heavily inspired by that and in the next part you will see how to use it to apply the package to behavior does that answer those questions oh yeah perfect definitely the functions look great and your team's understanding and Clarity around the problem setup taking that all the way through the helper functions for the matrices that's where so much of the friction is so this makes it really clear it's a great open source point it's also where a kind of future development will be also there so the API can be improved for sure and we will work on this also to make it because I think one of the main challenges active inference has faced as a tool for C modeling is that it's hard to use and hard to understand and that's why much of the mathematical psychology world for example just leaves it they say we have a two or three equation model why would we use your beam of of a mon machine instead but just as it has been the case with many other methods making tools that make it easy to use and

conceptualizations that make it easy to understand is the kind of thing that will make these models um feasible and therefore used more building yeah that's that's making me think like RX and fur is the car factory this is the bike and active inference is the wheel you can think of it yeah you need the logo yeah all right let's do the the last important section and then see how much time okay and because we're short in time I will need to skip through a few of the sections but I hope it's going to be all right yeah so I think Peter already mentioned that we the sort of environment of packages in Julia and one of them is uh action models which is sort of a sister package uh for us which mediates the data fitting process for us right so here we are essentially going in the opposite direction where our inputs and the the actions of our let's say experimental subjects uh are known but now we are interested in the hidden or latent uh parameters of the subjective um generative model um so this a sort of core component of action models is the action model function which depending on our on our model can be an arbitrary function which takes a single input which in our case is a single observation it updates the states of the agent and then it sort of quantifies the probabilities of actions so we end up with a probability distribution from which we can sample uh our actions right and so in our case we have this nice function called action pmf which is our active inference action model and it takes these two arguments which is our initialized aif object with the parameters of the generative model settings Etc and it takes the the takes a single observation and then the out it outputs the action distribution so in this particular case we have two action distributions one for each state Factor right and then another core component of action models is an agent so it sort of wraps our action model function and our um active inference um object in more more uh abstract structure which with the use will be will be using for fitting and for um simulation as well and it also stores parameters States uh history Etc right now I will just allow myself to skip through some of the convenience functions so I can just go straight to the data fitting if it's all right okay so um this is I would assume the most interesting interesting part of this so how to actually fit active inference uh models to data so um all all of our fitting functions kind of operate on touring and we wrap all of the necessary touring operations in our convenience functions for fitting so let's say for example we want to fit a single subject and we want to recover a single parameter this sort of minimal simple example so we start by specifying the prior so here let's say we want to recover our action Precision parameter Alpha so we set the prior for it let's say a gaussian with a mean of six and sigma 2 and set it in dictionary in any case we want to put more parameters at once we just add them to the same dictionary in their corresponding prior distributions and so now we can use the create model function

which will instantiate our probabilistic model on our data and it takes these uh inputs which is our initialized agents takes our priors and our collected observations and actions which we collect let's say from from a behavioral uh experiment and in in this case uh we're just using some data from simulated teams so here we um our actions um is a matrix which um which has rows corresponding to time steps or let's say trials and columns corresponding to State factors and similarly in observations uh we have columns corresponding to Observation modalities right and then we'll have our model object instantiated which we can easily fit in a single line using `fit_model` function which will run the sampler with appropriate defaults and then we can like obviously people would want to pass some more specific settings then to the sampler so here I'm passing for example uh the number of iterations per chain and I'm running four chains and then I'm using the distributed module to run the chains um in parallel right and then we after the sampling is finished then we will end up with the results object which which contains our chains and then we can work further with this right so here I'm just going to quickly go through some of our other convenience functions for example to rename chains which will rename the names of the parameters in our chain so something more more intuitive and we can extract quantities from our chains so we can just simply display the results of of our chain so we can get the summary statistics some information about about the conversions of our chains quantal Etc right or then we can just use the sort of native jly uploading functions we can pass our our chains to it uh to take a look at chain traces or posterior distributions or we can uh what we can do is we can sample from our prior so here I'm taking a th samples from the prior and I'm I'm using our `plot_parameters` function to plot um the the posterior distribution against um against the prior distribution um for our um synthetic participant let's say okay and this is a part is going to be mostly interesting I think for people in computational Psychiatry and cognitive modeling and it's the case where we want to fit uh full data set so in this case let's say that we have some data with 10 participants and let's assume that we have two clinical groups um in our data it can be I don't know um control and um whatever psychotic or it can be young um old um right and we can simply pass the wall data set with our collected um data let's say from from our behavioral experiment there same create model function so instantiates the the probabilistic model um on the world data set and then an additional things that we need to tell the function is which columns to use to differentiate between individual subjects and which columns uh contain our observations and which columns contain our actions and again we have our model instantiated on the W data set and we can just pass it on to to our `fit_model` function right and again this following is sort of

the same procedure as before we can play around with our recovered chains here we have the the recovered values of alpha for for each participant and I'm using um Native J functions to topl the chain traces and again we can let's say sample from the prior and quot the posterior against prior distributions for each participant right and then so of keeping also with the with this default procedure in cognitive modeling now let's say that we have our estimates and we want to see where there is actually a difference right in in our two groups right so here I'm just extracting the quantities from the chain and I'm getting the estimates which these case are medians and we can see here that there are indeed um two groups present in our data right and then after let's say that we run some statistics on our estimates then we might be also interested uh to see whether there's any correlation with other states of our agent and let's say some physiological data here I'm pling the state action prediction error reason why people want to correlate let's say with with brain data or pupilometry for example and uh for people who might want to use variational inference for uh various reasons over over sampling the good news is that touring can do variational inference to so here I just sort of remade the tutorial which you can find on the official ing um web page where I'm in the beginning just initializing the variational inference uh algorithm with a sort of default number of samples and a default number of iterations and importantly I'm setting the automatic differentiation back end to reverse mode automatic differentiation and then in the vi function which runs the variational inference algorithm I'm just reusing our probabilistic model uh which we have already created previously right so this is this is very nice and what we end up with is our variational posterior which is a tpe of distribution so we can sample from it or work further with it right and just to rather quickly wrap up so um we're currently working on the documentation uh we know it's important so um it should be done in the next few weeks let's say uh until then you can either Reach Out or uh you can refer to action models documentation which is uh really nice to wrate in okay and yeah just briefly just to summarize what we need to do in our in our um next potential future uh so we need to optimize the package is fast but there are still a lot of things that need to be improved then the idea was to for example uh be able to fit all parameters because for now we we're only able to to recover the the sort of hyper parameters or what been called the non-matrix parameters but in principle one can also one should be also able to fit an entire let's say a matrix or C Matrix which would be really nice and then okay we can um extend the parameter learning to all other uh parameters of the of the generative model and that idea was to introduce a sort of custom parameterizations where a user could introduce a sort of temperature par also on on on an a matrix and this could be also in principle then

recovered using data fitting from empirical data um and lastly I'm just going to say that um for our fitting we are using reverse automatic differentiation which is fast but we are also planning to start experimenting with other uh engines which are currently under development in Chula also which includes enzyme and moon cake which might be potentially uh faster and and then on the side of action models um Peter has been working for some time also on the possibility of adding uh linear regressions so the idea would be that we could fit the data we could recover our estimates and we could run linear regressions with our with our estimates or with with other um other variables that we might have in one model yes which is statistically optimal yes okay and the last thing we're going to hear from is that uh when you decide that you want to start experimenting with our package and you encounter any issue or do you have any specific feature in mind that you wouldd like us to implement please feel free to your open issue uh any kind of feedback will be greatly um appreciated thank you and we're out of time so I'll I'll just briefly mention this point that you can treat a whole group of Agents as one agent and do c modeling art it's a separate paper so I'm sure there'll be time to talk about that at another time to reach out if that's interesting as well um yes I think that's it thanks Soul this is a huge Advance it's really awesome brings a lot of features a lot closer to people's hands and minds and and it's exciting that that we'll continue developing and learning about it thank you okay thank you till next time time